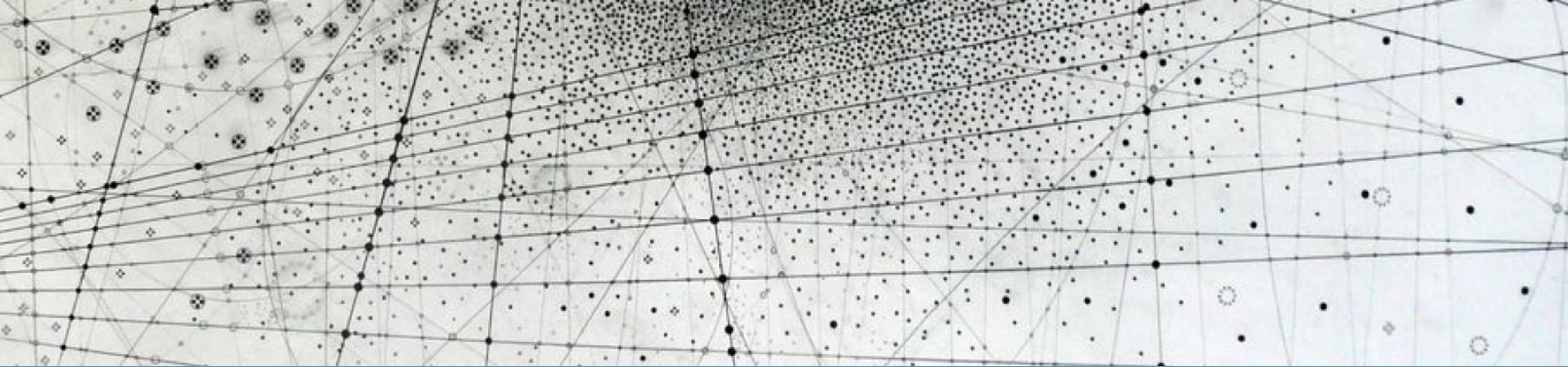


Research Methods in Machine Learning & Physics

Pset 1, Problem #10

Problem 10: Detecting a hidden signal

Challenge: Finally, let's see if we can use this NN to find a hidden signal in the data. We will construct a dataset with a mystery mass peak comprising two taus that is injected to background events at a level of 1.5% (i.e. 750 events in 50,000). Given the output variables of the two taus, find the invariant mass of the hidden signal. Report your answer as a number with precision `1e1`.

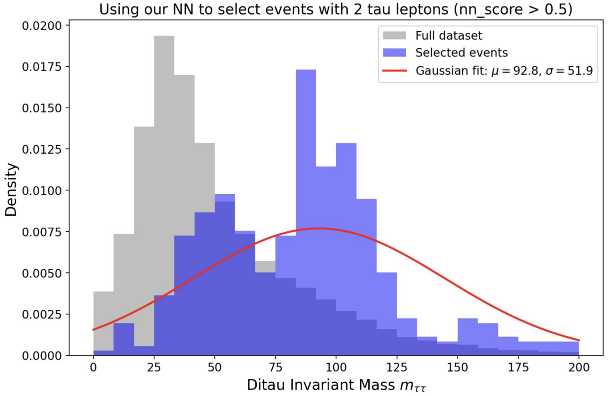
[ ]:
```
### use pip to install pylorentz
```

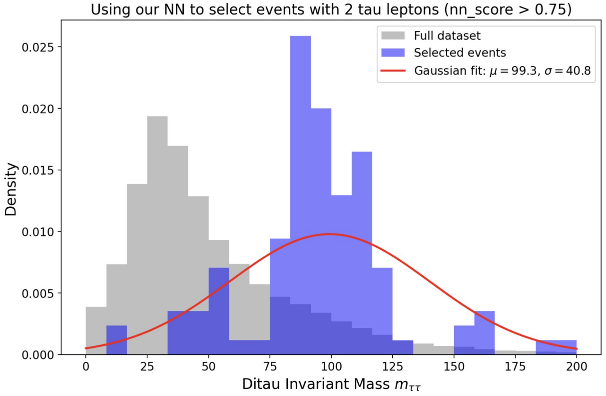
[ ]:
```
from pylorentz import Momentum4

def calculate_invariant_mass(vector_1, vector_2):
    tau_1 = Momentum4.m_eta_phi_pt(vector_1[3], vector_1[1], vector_1[2], vector_1[0])
    tau_2 = Momentum4.m_eta_phi_pt(vector_2[3], vector_2[1], vector_2[2], vector_2[0])
    return (tau_1+tau_2).m

def make_dataset(mass_function, dataset):
    ### apply some basic quality cuts:
    mask=(dataset["pt2"].array() > 0)
    mask=(abs(dataset["eta2"].array()) < 2.1) & mask
```

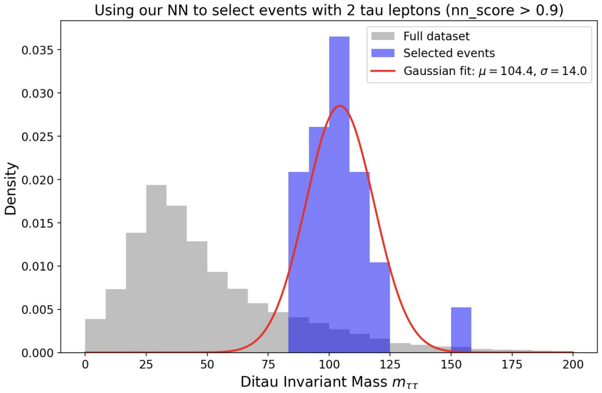
Pset 1, Problem #10

Basic idea: apply the same NN to each of the two input taus and make increasingly harsh cuts on both.

```
tau1_scores = model(torch.tensor(in1_mix)).detach().cpu().numpy()
tau2_scores = model(torch.tensor(in2_mix)).detach().cpu().numpy()
```







Quick recap

Quick Recap: Likelihood ratio trick

You can re-use a NN classifier to approximate a likelihood ratio between two distributions.

$$\mathcal{L}_{\mathrm{BCE}}[f] = -\int d\vec{x}\,\left(p_A(\vec{x})\log(f(\vec{x})) + p_B(\vec{x})\log(1-f(\vec{x}))\right)$$

Find f that minimizes this functional
(or approximately minimizes it)

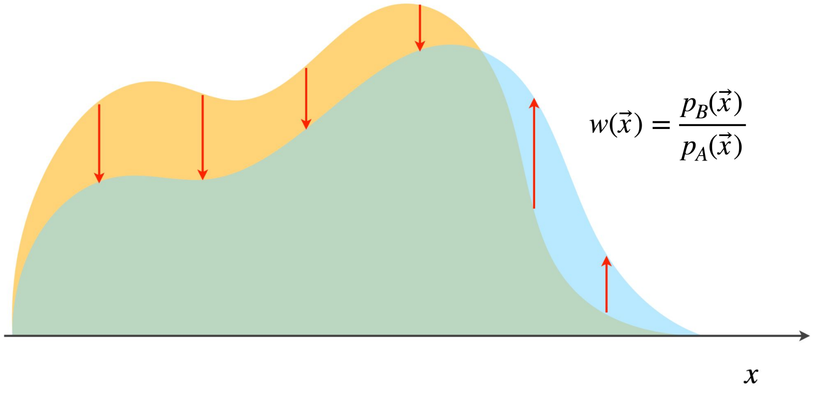
Rescaling of classifier output $$\frac{f(\vec{x})}{1-f(\vec{x})} = \frac{p_A(\vec{x})}{p_B(\vec{x})}$$ Likelihood ratio

Quick Recap: Likelihood ratio trick

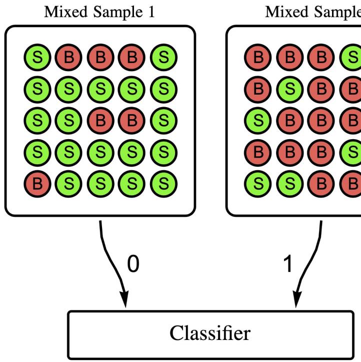
Rescaling of classifier output $$\frac{f(\vec{x})}{1-f(\vec{x})} = \frac{p_A(\vec{x})}{p_B(\vec{x})}$$ Likelihood ratio

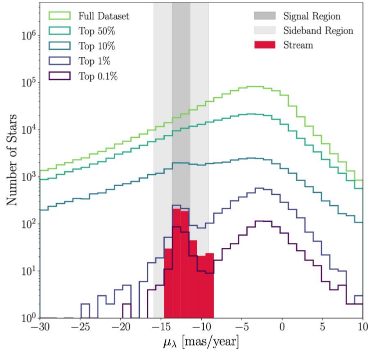
What can you do with a likelihood ratio?

Reweight Distribution A → Distribution B



Search for anomalous bumps in mixtures of classes





Hyperparameter optimization

Definition of a hyperparameter

A **hyperparameter** refers to anything that <u>we</u> control that affects the learning process.

Classic examples: the learning rate, dropout rate, # of epochs, etc.

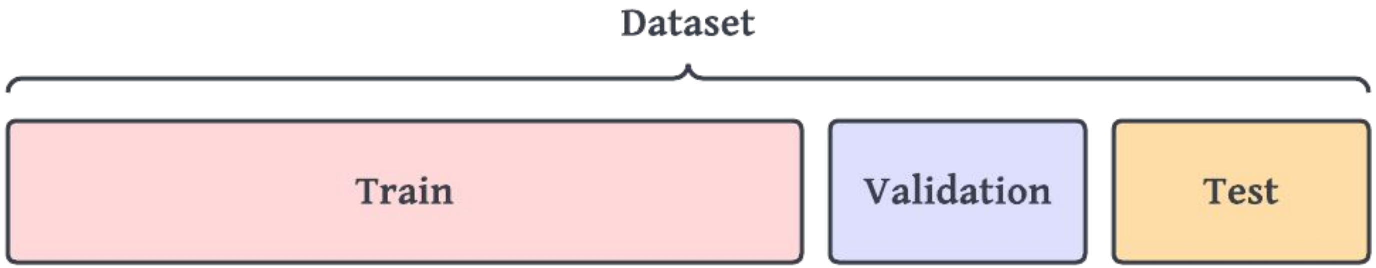
Choosing the right (or wrong) hyperparameters can heavily constrain what your model is capable of learning in practice.

Hyperparameter tuning

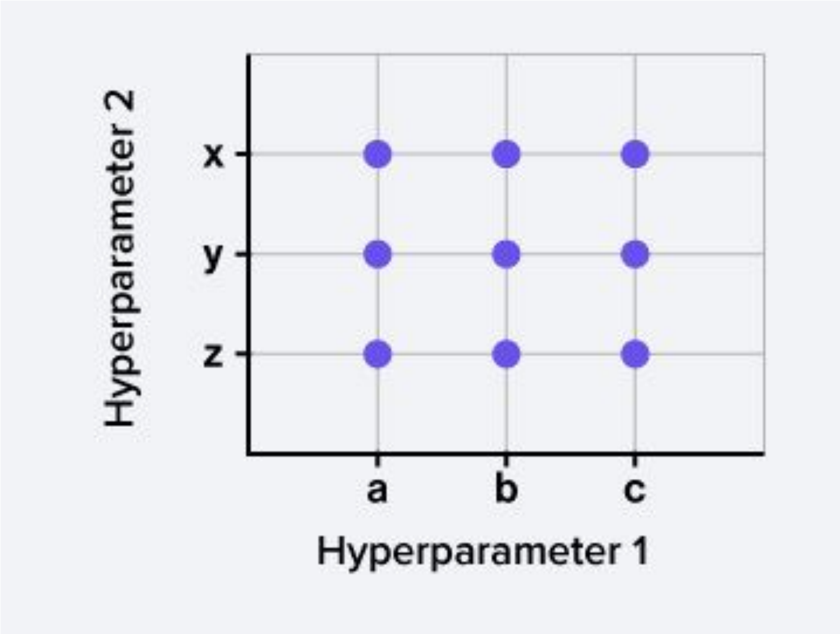
How do we know what hyperparameters are best? There might not be strong theoretical grounding to inform the best practices on a particular dataset.

You don't want to use your training data directly – you want to save that to train your optimized model. And you need to preserve a holdout **test set** for reporting your results.

Solution: make a validation set!



Strategy: grid search



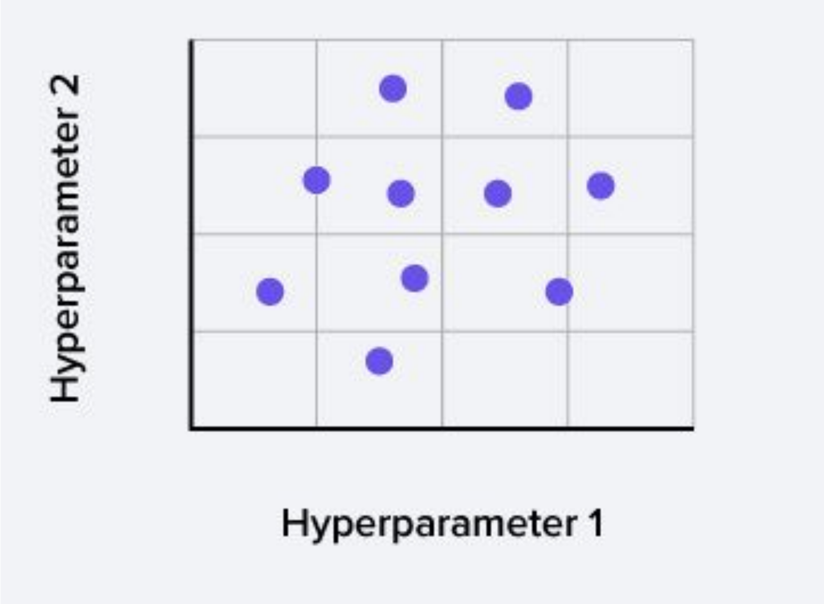
Pros:

- Can exhaustively search a given space
- Good for finding regions of interest
- Reproducible

Cons:

- Computationally expensive
- Might miss critical regions

Strategy: random search



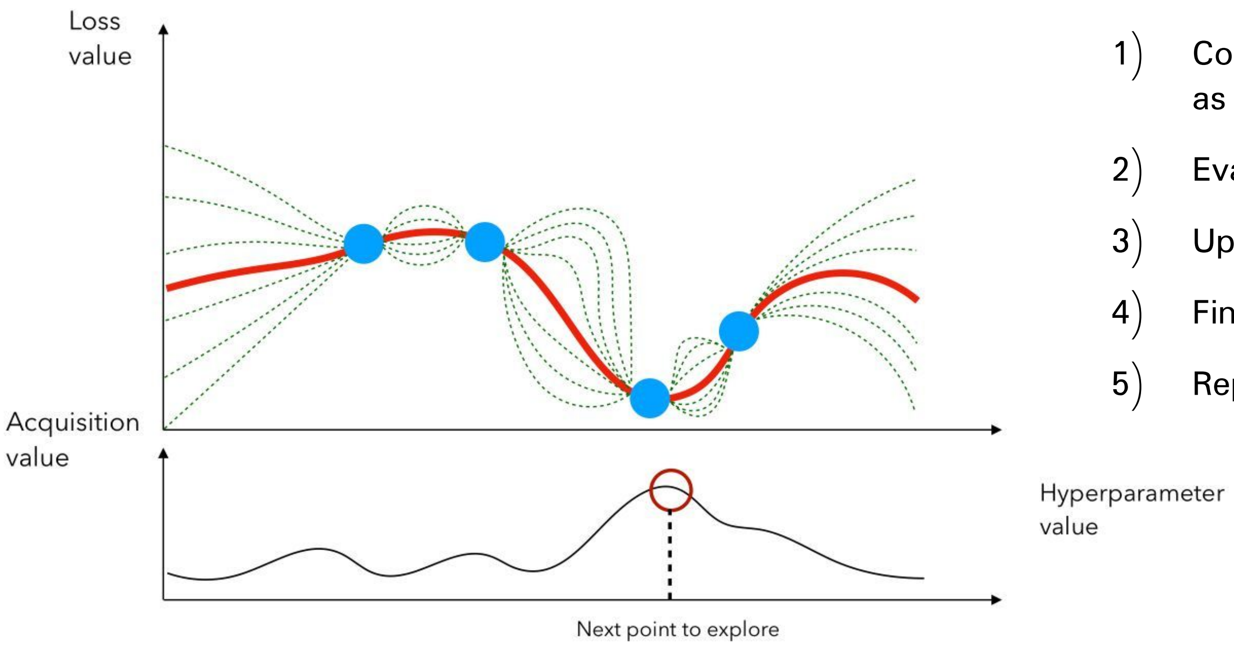
Pros:

- Much more efficient than grid search
- Usually "good enough" to be a default

Cons:

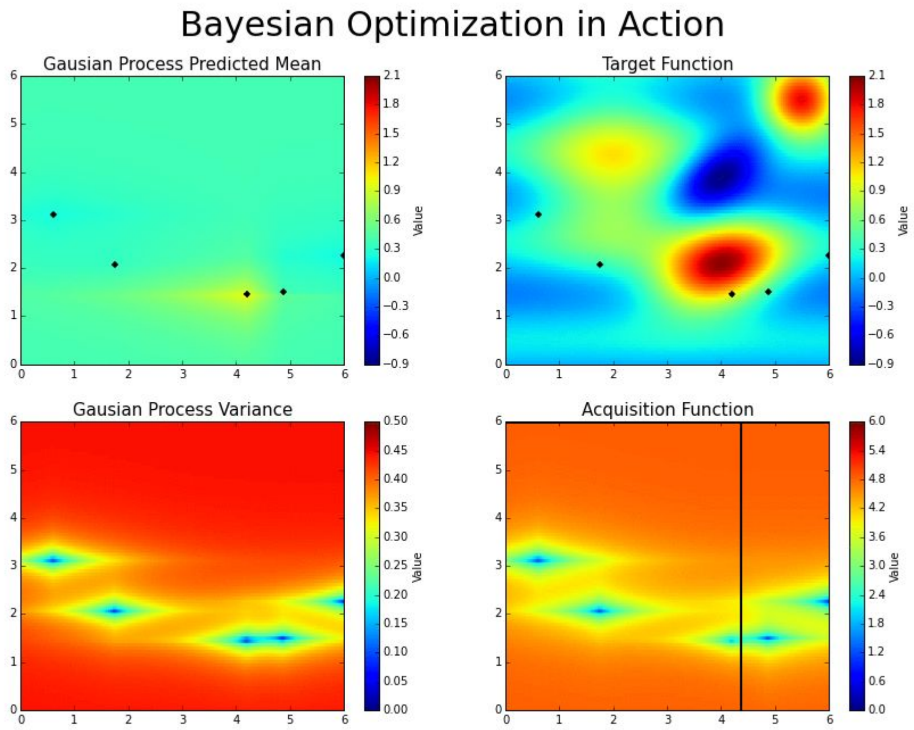
- Not exhaustive
- Not reproducible

Strategy: Bayesian Optimization



1) Construct a surrogate model for the loss as a function of each hyperparameter
2) Evaluate at a few points along this curve
3) Update your surrogate model
4) Find a new point to explore
5) Repeat

Strategy: Bayesian Optimization



1) Construct a surrogate model for the loss as a function of each hyperparameter
2) Evaluate at a few points along this curve
3) Update your surrogate model
4) Find a new point to explore
5) Repeat

Which hyperparameters to test?

Batch size should generally not be tuned to optimize validation performance.

Any such differences in the validation performance will generally go away once you individually tune the other hyperparameters for each given batch size.

But it is critical to understand trade-offs in training time & compute resources!

- **A rule of thumb:** increase your batch size by powers of 2 until you run out of memory on a GPU, then choose the one just below that.

Other critical ones:

- **Regularizer-related** (dropout, L2 normalization, patience for early stopping, etc.)
- **Optimizer-related** (learning rate, etc.)
- **Training-related** (number of epochs, etc.)

Additional resources

https://github.com/google-research/tuning_playbook

A scientific approach to improving model performance

For the purposes of this document, the ultimate goal of machine learning development is to maximize the utility of the deployed model. Even though many aspects of the development process differ between applications (e.g. length of time, available computing resources, type of model), we can typically use the same basic steps and principles on any problem.

Our guidance below makes the following assumptions:

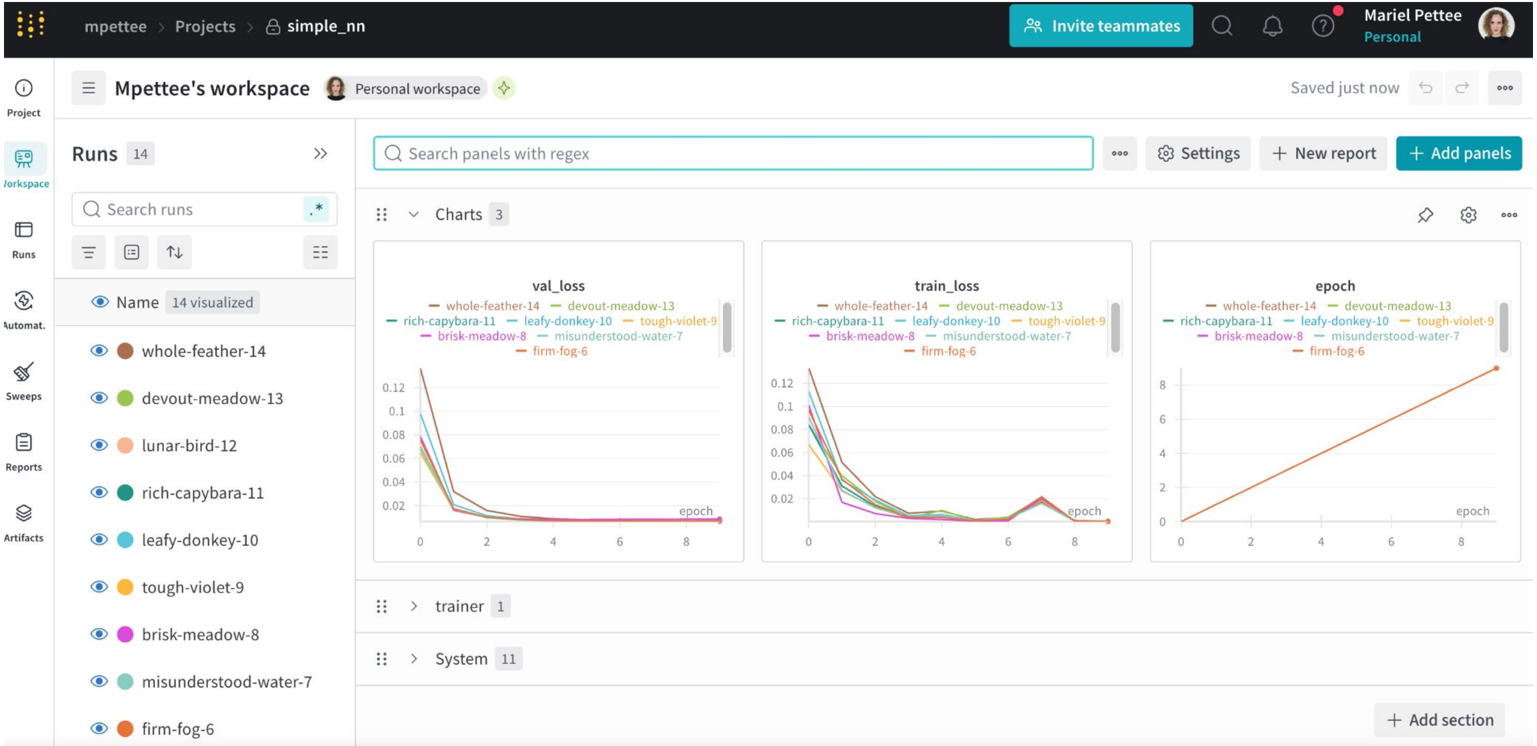
- There is already a fully-running training pipeline along with a configuration that obtains a reasonable result.
- There are enough computational resources available to conduct meaningful tuning experiments and run at least several training jobs in parallel.

The incremental tuning strategy

Summary: *Start with a simple configuration and incrementally make improvements while building up insight into the problem. Make sure that any improvement is based on strong evidence to avoid adding unnecessary complexity.*

Additional resources

wandb.ai



In-class work

- Hyperparameter optimization in Weights & Biases (wandb.ai)
  - Use a model & dataset of your choice (could be from the problem set or previous in-class work)
  - Make an account in Weights & Biases
  - Either in a notebook or in the command line, make a new project & submit several training runs
  - Look at the validation loss curves with log scale on the y axis & different levels of smoothing
  - Explore “Runs” vs. “Workspace” mode
  - Explore “Group by” and other filtering strategies
  - Submit a hyperparameter sweep! (Grid search, random search, & Bayesian)
  - Generate a report
- PS: See me if you are not on the Overleaf for our collective glossary